# P7 Report

## Task 1

### Divide by 3

```
int div_3(int arr[], int n) {
    // Counts the number of values in the array that are divisible by 3.
    // The length of arr is n.
    int count = 0;
    for (int i=0; i<n; i++) {
        // Your invariant should apply here!
        if (arr[i] % 3 == 0) {
            count ++;
        }
    }
    return count;
}
```

Invariant: At the beginning of each iteration, count is the number of values divisible among the first i elements in the array.

### Sum of primes

```
int sum_of_primes(int n) {
    // Computes the sum of the first n primes.
    int total = 0;
    for (int i=1; i<n; i++) {
        // Your invariant should apply here!
        if (is_prime(i)) { // You may assume this function exists and
works.
            total = total + i;
        }
    }
    return total;
}
```

Invariant: At the beginning of each iteration, total is the number of all prime numbers less than i.

### Shaker Sorting

```
void shaker_sort(int arr[], int n) {
    // Sorts the list of arr.
    // The length of arr is `n`.
    int i = 0;
    int j = n - 1;
    while (i < j) {
        // Your invariant should apply here!
        // Selects the index of the smallest item from arr[i:n]
        int small = smallest_item_index(arr, n, i);
        swap(arr[i], arr[small]);
        // Selects the index of the largest item from arr[0:j+1]
```

```
        int large = largest_item_index(arr, n, j);
        swap(arr[j], arr[large]);
        i++;
        j--;
    }
}
```

Invariant: At the beginning of each loop iteration, the first i elements are the i smallest elements in their correct sorted positions, while the elements after index j are the n-1-j largest elements in their correct sorted positions.

# Task 2

**Invariant Proof for function div_3**: At the beginning of each iteration, count is the number of values divisible by 3 among the first i elements of the array.

**Initialisation**: The first time the invariant is reached, i = 0 and count = 0.

Since none of the array elements have processed yet, the first 0 elements contain 0 values divisible by 3. Therefore, count correctly stores the number of values divisible by 3 among the first i = 0 elements.

Thus, the invariant holds before the first iteration.

**Maintenance:** Let k be an integer such that 0<= k <n. Suppose the invariant holds for i = k. That is count currently holds the number of values divisible by 3 among the first k elements of arr.
Then, we execute line 7 – one of two scenarios occurs:

**Case1: arr[k] is divisible by 3**
Then arr[k] % 3 == 0 is true, so the count is incremented by 1. Since count previously stored the number of divisible values among the first k elements, it now stores the number of divisible values among the first k+1 elements.
**Case2: arr[k] is not divisible by 3**
Then arr[k]% 3 == 0 is false, so the count is not changed. Since arr[k] does not add another value divisible by 3, count still correctly stores the number of divisible values among the first k+1 elements.

In either case, count currently holds the number of values divisible by 3 among the first k+1 elements.

Then, we execute the loop iteration code i++, in other words, we set i =k+1.

Then, the invariant line is researched again. We know that:
count currently holds the number of values divisible by 3 among the first k+1 elements.
i = k+1

So we have: the count currently holds the number of values divisible by 3 among the first k+1 = i elements

And so the invariant holds.

**Termination**

At the final occurrence of the invariant, before loop exit, we have i=n. From our maintained invariant, this implies count currently holds the number of values divisible by 3 among the first n elements. In other word, count holds the whole number of values divisible by 3 among the whole elements. As such, div_3 computes the correct value.